

Confirmed clean state on the correct commit. Now let's map every current mutation path for Papers, Journeys, and Publishing state — this is the core fact-finding for the design. This is a critical finding — let me confirm these six handlers are the only mutation entry points (no component bypasses them with direct setPapers/setJourneys access). Confirmed — a single choke point exists for Papers/Journeys (all nested edits construct a full object and call onUpdateJourney/onUpdatePaper), but Publishing state is architecturally different — entirely local to one component with no equivalent centralization. Let's verify Papers follows the same pattern, then inspect the import/export mechanisms. I now have sufficient grounded facts. Working tree confirmed clean throughout. Producing the design proposal. # Stage 3 Live WorkProduct Architecture Proposal

Design only. No files modified, no commits created — verified clean throughout and at the end of this task.

1. Repository / Governance Confirmation

FACT — Branch stage2/workproduct-schema-and-migration, HEAD
075e6fb93b11184d07583f08b0f36a2adeaaa2ae, parent
746eed7d41a1256e4b5e28f0f66ad39de7b75f14. Working tree clean before and after this task.

FACT — docs/ROADMAP.md current_stage: 1, untouched.

CONTRACT — Per D-005, only an authorised human changes current_stage; this document does not attempt to.

FACT — Stage 2 is frozen and unmodified during this task (verified: no diff against 075e6fb).

2. Current Architecture (FACT)

Papers and Journeys — already centralized, single choke point:

App.tsx holds papers/journeys as useState, and exactly 6 handler functions own every mutation: handleUpdatePaper, handleAddPaper, handleDeletePaper, handleUpdateJourney, handleAddJourney, handleDeleteJourney (plus two bulk operations, handleResetAllData/handleRestoreDemoData). Verified exhaustively: every setPapers/setJourneys call in the entire codebase occurs inside these functions in App.tsx — no other file calls the setters directly. At least 8 distinct components (CitationEngine, CreativePublishingWorkspace, LiteratureIntelligence, LiteratureLibrary, ResearchIntelligenceLayer, ResearchWorkspace, FundingWorkspace, ResearchTimeline) trigger these mutations via props (onUpdatePaper, onAddPaper, etc.), but all resolve to the same 6 functions. Nested mutations already flow through the same choke point. Chapter content edits, task changes, timeline changes, and linked-paper changes are all implemented by constructing a complete, updated ResearchJourney object and calling onUpdateJourney(fullObject) — never a separate nested-only mutation path. Verified in ResearchWorkspace.tsx. The paper-import feature (DataIngestionModule.tsx, via onIngestPapers) also resolves to handleAddPaper per paper — bulk import already uses the same choke point. Collection[] is a useState initialized from INITIAL_COLLECTIONS with no

setter ever used (`const [collections] = useState(...)`) — it is static demo data, not a live, mutable, or persisted entity today.

Publishing — architecturally different, no equivalent centralization:

Publishing state (`docTitle`, `draftContent`, `outline`, `notes`, `checklist`, `importedDocs`, plus two read-only-in-practice fields) lives entirely inside `CreativePublishingWorkspace.tsx` as 6 independent `useState/useEffect` pairs, each persisting directly to its own `pub_*` key. Verified: no other file references these setters. Publishing state is not lifted to `App.tsx` at all.

Import/export — narrower than expected, and doesn't touch substantive work today:

`Settings.tsx`'s `.rcp` export bundles only profile fields, `daily_focus`, `wellbeing_small_wins`, `draft_companion_text` (a distinct key from `WritingCompanion.tsx`, separate from `pub_draft_content`), `scholar_project_type`, accessibility settings, and `scholar_feedback_logs`. It does not include `Papers`, `Journeys`, or any `pub_*` key. There is no import/restore function at all — no `FileReader`, no file-upload handler for `.rcp`. "Restore original demos" only resets to the static `INITIAL_PAPERS/INITIAL_JOURNEYS` seed constants; it is not a user-file restore.

Stage 2 artifact: `peessoa_work_products`, re-derived from legacy state on each app load by `runWorkProductMigration()`, called once in `App.tsx` as a side effect. Not read by anything else.

3. Architectural Problem (why the migration output can't yet be the live read source)

The Stage 2 migration transforms legacy state into `WorkProduct[]` on each load, but nothing transforms mutations back. Since §2 confirms `Papers/Journeys` funnel through exactly 6 `App.tsx` functions that call `setPapers/setJourneys` (writing only to `scholar_papers/scholar_journeys`), any edit made after the initial load is invisible to `peessoa_work_products` until the next reload re-runs the migration. If the UI read from `peessoa_work_products` as primary, a reload immediately after an edit would show correct data (migration re-derives fresh each time) — but within a session, before any reload, `peessoa_work_products` would already be stale relative to live state. Worse, if a future feature ever read `peessoa_work_products` mid-session (e.g. a Stage 3 project view), it would silently show outdated content. This isn't a bug in the migration — it did exactly what a one-time/per-load transformation is supposed to do — it's a structural gap: no mutation path currently writes to the new representation.

4. Alternatives A — `WorkProduct[]` canonical B — Domain state canonical, `WorkProduct` as adapter C — Dedicated domain stores + sync layer Consistency High — single state, single write path, no reconciliation needed Medium — `WorkProduct[]` still has to be derived somewhere; if derived only at persistence-write time, it's consistent, but conceptually `WorkProduct` never becomes "the" model, just an export format Low-Medium — two live representations by construction, with a sync layer whose entire job is preventing exactly the divergence problem in §3 Simplicity High, given §2's finding — only ~6-8 functions need touching, not a broad refactor High for `Papers/Journeys` (barely changes anything); leaves the Product Contract's "WorkProduct as canonical representation" aspiration unrealized Low

— a synchronisation layer is new infrastructure with its own bugs (this is precisely the "separate synchronisation process" the brief says to avoid) Type safety High — hybrid envelope already preserves typed payloads (Stage 2 design) High (payloads never move) High individually, but the sync layer is a new place for type/shape drift between the two stores Migration path Direct continuation of Stage 2 output — no rework Requires deciding Stage 2's WorkProduct[] was "just for show" Requires building what Stage 2 deliberately avoided (a bookkeeping/sync mechanism) Component impact Low, given §2 — components already receive Paper/ResearchJourney via props from App.tsx; if App.tsx derives those same props from canonical WorkProduct[] via selectors, components need zero changes None None, but the new sync layer is a new component-scale piece of infrastructure Persistence coupling Low — one adapter, one canonical shape Medium — still two shapes reaching storage (legacy keys + derived WorkProduct[]) unless legacy writing is also retired, which is really Option A by another name High — both stores need persistence, plus sync state Risk of stale data Eliminated by construction — one state, one write path Present whenever anything reads WorkProduct[] before the next full derivation High — the exact failure mode in §3, deliberately engineered around rather than solved Implementation complexity Low-Medium (touch ~6-8 functions + 1 new publishing choke point + 1 adapter) Low, but doesn't achieve the stated Stage 3 objective High Compatible with Stage 4 Yes — adapter boundary is the seam Stage 4 needs Yes, trivially (nothing changed) Yes, but inherits the sync layer's complexity into Stage 4 too Compatible with Product Contract High — directly realizes "Work Product... produced, developed or maintained through the person's work" as the thing the app actually operates on Weak — Contract implies WorkProduct is meaningful, not just an export shadow Medium — technically compatible, but adds machinery the Contract doesn't ask for Risk of premature architecture Low — §2 shows the mutation surface is already narrow; formalizing it isn't inventing new structure, it's making explicit what already exists Low, but avoids the actual objective High — building sync/conflict-adjacent infrastructure the task explicitly says not to build Recommendation: A — WorkProduct[] becomes canonical

This is justified by §2's facts, not the brief's suggestion: the mutation surface for Papers and Journeys is already a narrow, 6-function choke point, and nested mutations already flow through full-object replacement. Converting that existing choke point to update canonical WorkProduct[] state is a small, mechanical change — not the invention of new architecture. Option C would be solving a problem (divergent live stores) that Option A avoids needing to solve at all. Option B fails to make WorkProduct genuinely authoritative, leaving the Stage 2 → 3 objective unmet.

5. Recommended Architecture — Data Flow User action (via existing component props/handlers) ↓ App.tsx mutation functions (same 6-8 functions as today, extended) ↓ Canonical WorkProduct[] state (single source of truth, in App.tsx) ↓

↓ ↓ ↓ Persistence Typed selectors (future: project adapter (Paper[], Research resolution, Stage 3+ ↓ Journey[], Publishing later work) localStorage DraftPayload) fed to (pessoa_work_ existing components, products) UNCHANGED prop types

Existing components continue receiving Paper[]/ResearchJourney[]/publishing fields exactly as today — they do not need to know WorkProduct exists. Only App.tsx (and, for publishing,

CreativePublishingWorkspace.tsx's own state) changes what it stores internally and how it persists.

6. State Model

PROPOSAL — WorkProduct[] lives as a single useState in App.tsx, replacing the separate papers/journeys useState calls (publishing's local state is discussed separately below, §7). Typed selectors derive the existing shapes components already expect:

```
const workProducts = useState<WorkProduct[]>(…); const papers: Paper[] = useMemo( () => workProducts.filter((wp): wp is WorkProductPaper => wp.kind === 'paper').map((wp) => wp.payload), [workProducts] ); const journeys: ResearchJourney[] = useMemo( () => workProducts.filter((wp): wp is WorkProductResearchJourney => wp.kind === 'research_journey').map((wp) => wp.payload), [workProducts] );
```

Components keep receiving papers/journeys exactly as now — zero component-level changes, per §2's finding that they only ever consume the payload shape, never the envelope. Kind-narrowing happens in exactly one place: these two selectors in App.tsx. This directly answers the brief's question ("where should narrowing occur?") — nowhere else needs to.

7. Mutation Model — mapping every existing path

Because §2 established the mutation surface is already narrow, the mapping is direct:

Existing mutation Current effect Proposed effect
handleUpdatePaper(updated) setPapers(prev => prev.map(...)) setWorkProducts(prev => prev.map(wp => wp.kind==='paper' && wp.id===updated.id ? {...wp, payload: updated, updatedAt: now} : wp))
handleAddPaper(added) setPapers(prev => [...prev, added]) append a new WorkProductPaper envelope (id: added.id, createdAt: now, updatedAt: now, payload: added)
handleDeletePaper(id) setPapers(prev => prev.filter(...)) filter matching kind:'paper', id out of workProducts
handleUpdateJourney / handleAddJourney / handleDeleteJourney analogous, on kind:'research_journey' analogous — and because nested chapter/task/timeline edits already arrive as a full replaced ResearchJourney via onUpdateJourney, no additional nested-mutation handling is needed
DataIngestionModule bulk import loops onAddPaper per item unchanged — still loops the same handler, which now updates canonical state
handleResetAllData / handleRestoreDemoData reset papers/journeys to seed constants
reset workProducts to seed-derived envelopes (wrap INITIAL_PAPERS/INITIAL_JOURNEYS once, at reset time)
Publishing (docTitle, draftContent, outline, notes, checklist, importedDocs) 6 independent local useState/useEffect pairs in CreativePublishingWorkspace.tsx
PROPOSAL: keep the component's local useStates exactly as-is (no lifting, no prop-drilling change — avoiding the component restructuring the brief warns against), but replace the 6 independent useEffects with one combined effect that assembles the current values into a single publishing_draft WorkProduct and updates canonical state. This is the "minimum necessary architectural change" for publishing: one file, one effect consolidated from six, no state relocation.

This satisfies "map every current mutation path" (§ Mutation architecture) using only the mutation surface §2 already found — no new entry points are invented.

8. Persistence Adapter

PROPOSAL — a thin module (natural home: replacing/absorbing `src/lib/workProductMigration.ts`'s write responsibilities) with exactly two operations:

```
function loadWorkProducts(): WorkProduct[] // reads pessoa_work_products function
saveWorkProducts(items: WorkProduct[]): SaveResult // writes pessoa_work_products
```

`App.tsx` calls `saveWorkProducts` in a `useEffect` keyed on `workProducts` (replacing today's `useEffects` that write `scholar_papers/scholar_journeys` directly) — the same pattern already in use, just retargeted at one key instead of two. The adapter's only job is the `localStorage` read/write and defensive parsing (already implemented in Stage 2's `readWorkProducts/write` logic) — it has no knowledge of kind, Paper, or ResearchJourney. This is what makes Stage 4 possible without redesigning the application model: swapping `localStorage.getItem/setItem` for `indexedDB` calls inside this one module changes nothing above it.

9. Legacy-Key Transition

PROPOSAL — a clean cutover, not permanent dual-writing, consistent with the brief's stated preference and justified by §2/§6 of the current app:

Cutover point: the moment `App.tsx` switches its `useState/useEffect` pair from `papers/journeys` (writing `scholar_papers/scholar_journeys`) to `workProducts` (writing `pessoa_work_products`), the legacy keys stop being written to. This is a single, explicit code change, not a gradual/flag-driven migration. Legacy keys are not deleted at cutover (per D-009's spirit, extended) — they remain on disk as a stale historical snapshot from before cutover, readable for manual recovery/debugging, but the running application never reads or writes them again after cutover. Given §6's fact that current data is sample/demo content, there is no need for a fallback/dual-read period "just in case" — the risk being protected against (real user data loss) doesn't exist yet. This is explicitly the right moment to do a clean cutover rather than design a gradual one. OPEN DECISION (flagged in §14): should the one-time transition itself re-run `runWorkProductMigration()` once more immediately before cutover, to sweep up any last legacy edits made in the final Stage-2-only session before Stage 3 code ships? Recommended: yes, trivial and safe, since the migration function already exists and is idempotent.

FACT-grounded PROPOSAL: since §2 established the existing `.rcp` export already excludes `Papers/Journeys/Publishing`, and there is no import function at all, Stage 3's minimum obligation is narrow: the export function should be extended to include the canonical `WorkProduct[]` (a straightforward addition — `workProducts: workProductsRef` in the exported JSON), so the backup file actually captures substantive work for the first time. Building a corresponding import function (reading a `.rcp` file back in) is a real, user-facing feature that doesn't exist today in any form — it is not a Stage 3 "wiring" task but new functionality, and per the brief's own instruction ("do not implement unless explicitly authorised later"), this is

flagged as out of scope for this transition and listed as an open decision, not silently built or silently declined.

11. Failure and Consistency Model

How staleness is prevented: by construction, not by reconciliation. There is exactly one state (workProducts) and exactly one thing derives from it for reading (the papers/journeys selectors) and exactly one thing writes it to disk (the persistence adapter). There is no second store that could disagree with it, so "WorkProduct says A, legacy state says B" cannot occur post-cutover — there is no more "legacy state" being live-read anywhere.

Multiple mutation paths: all still resolve to the same setWorkProducts call inside React's state update queue — React's batching guarantees this remains a single, ordered stream of updates, same guarantee the current setPapers/setJourneys already relies on.

Asynchronous persistence / reload: the persistence useEffect fires after each workProducts change, same timing model as today's papers/journeys effects. A reload reads whatever was last successfully written — identical risk profile to today (a reload immediately after a change that hasn't yet flushed to localStorage could theoretically miss that last change; this is a pre-existing characteristic of the current app, not something Stage 3 introduces or worsens).

Failed persistence — PROPOSAL: per §12 of the brief, if saveWorkProducts throws (e.g. quota exceeded), the in-memory workProducts state has already updated (the mutation succeeded from the user's point of view), but the write failed. The adapter should return a result the caller can check ({ ok: boolean, error?} — matching Stage 2's existing console.warn-on-failure pattern, extended to also surface a toast/notice to the user via the same triggerToast mechanism already used elsewhere in the app for user-facing feedback) rather than silently claiming success. This is the minimum honest behaviour: don't build a retry queue or offline-write buffer (that's infrastructure the brief says not to invent), just don't lie about the outcome.

Partial writes: JSON.stringify + a single setItem call is atomic from the application's perspective (localStorage doesn't support partial writes within one setItem), so this class of failure reduces to the same "write failed, was reported" case above.

Imports: out of scope per §10; whenever built, it should produce WorkProduct[] entries directly (matching §10's framing) rather than legacy-shaped objects requiring a second migration pass.

12. Testing Plan (design only, not implemented)

State: creation/update/deletion of each kind via the App.tsx handler functions; kind-narrowing selector correctness (papers/journeys derived arrays match expectations); payload preservation across an update (no field silently dropped by the envelope-update spread).

Persistence: adapter write-then-read round-trip; reload behaviour (fresh App.tsx mount reads what was last written); malformed persisted data (reusing Stage 2's existing defensive-read

tests as the baseline, since the adapter's read side is the same logic); write failure (mock `localStorage.setItem` throwing, assert the failure is surfaced, not swallowed).

Consistency: mutate → assert workProducts updated → assert persisted value matches → simulate reload → assert identical state; assert no residual read of `scholar_papers/scholar_journeys` occurs anywhere post-cutover (a static/grep-style check, mirroring how this proposal itself verified the current mutation surface).

Regression: re-run the existing Stage 2 migration test suite unchanged (it tests the pure parse functions, which remain valid) and the existing P0 regression suite unchanged — this transition touches only `App.tsx`'s internal state wiring and `CreativePublishingWorkspace.tsx`'s persistence effect; it does not touch `server.ts` or any AI routing code at all.

13. Stage Boundary

Belongs to Stage 3 (this transition): canonical `WorkProduct[]` state in `App.tsx`; the mutation-function updates in §7; the persistence adapter in §8; the legacy-key cutover in §9; extending export (not import) per §10.

Explicitly NOT this transition / later stages:

Stage 4: IndexedDB, storage-engine migration, storage durability redesign — the adapter boundary in §8 exists specifically so Stage 4 doesn't touch anything above it. Stage 5: PWA — not addressed, not needed here. Stage 6: Tauri / native desktop / local-AI runtime architecture — not addressed. Project-centred UI, project-resolution logic, `projectId` population — remains deferred per D-002/D-007; this proposal only confirms `projectId` continues to exist as an unpopulated optional envelope field, unchanged from Stage 2. Import functionality (§10) — flagged, not built. Any P0 routing/privacy change — none proposed; `WorkProductPrivacyAnnotation` continues to exist as an optional field, populated only if/when a future feature attaches AI-processing provenance to a `WorkProduct`; no P0 code is touched by this proposal.

14. Human Decisions # Question Options

Recommendation Consequence 1 Should the cutover run one final

`runWorkProductMigration()` pass immediately before switching `App.tsx` to read/write

`workProducts` canonically, to catch any last-minute legacy edits? (a) Yes, one final sweep ·

(b) No, cut over from whatever `peessoa_work_products` already holds (a) Negligible cost;

avoids losing an edit made in the last pre-cutover session 2 Should `.rcp` export be extended

now to include `WorkProduct[]` (§10)? (a) Yes, as part of this transition · (b) Defer to a

separate task (a) — small, additive, directly fixes a pre-existing gap (export currently omits

all substantive work) (b) leaves backups meaningless for actual research work indefinitely 3

Should a corresponding import function be built (reading a `.rcp/WorkProduct` file back in)?

(a) Build it now, in this transition · (b) Treat as separate, later-authorised work (b) — it's new

functionality with no existing precedent to extend, not a wiring task Choosing (a) would

expand this transition beyond "make `WorkProduct[]` canonical" into shipping a new

user-facing feature 4 Should legacy keys (`scholar_papers`, `scholar_journeys`, `pub_*`) be

actively deleted at cutover, or merely stop being written to (§9)? (a) Stop writing only, leave

on disk · (b) Delete at cutover (a) (b) is irreversible for no real benefit given §6's fact that

current data is disposable sample content anyway — but deleting still isn't necessary to

achieve the architectural goal, so there's no reason to take an irreversible action 5 Should

WorkProductPrivacyAnnotation (existing optional field) start actually being populated for any WorkProduct in this transition? (a) No, leave unpopulated as today · (b) Start populating it for some kind (a) Nothing in current mutation paths produces AI-provenance-bearing WorkProducts yet (per D-010, AI results still aren't persisted) — populating it now would have nothing real to populate it with

Final gate confirmed: no source files changed, no schema/migration/persistence/UI changes, no legacy-key changes, no commit created. Confirmed: working tree clean, HEAD unchanged at 075e6fb. This is the complete design deliverable — awaiting review before any implementation begins.